Part III: Dynamic Programming

Lecture 11: Chain Matrix Multiplication

Computer Science and Technology
United International College

Objective and Outline

Objective: Another example of dynamic programming
Reference: Section 15.2 of CLRS
Outline:

1. Review of matrix multiplication
2. The Chain Matrix Multiplication Problem
3. A dynamic programming algorithm for chain matrix multiplication

Review of Matrix Multiplication

Matrix: An $n * m$ matrix $A = [a[i, j]]$ is a two-dimensional array

$$A = \begin{bmatrix} a[1,1] & a[1,2] & \dots & a[1,m-1] & a[1,m] \\ a[2,1] & a[2,2] & \dots & a[2,m-1] & a[2,m] \\ \dots & \dots & & \dots & \dots \\ a[n,1] & a[n,2] & \dots & a[n,m-1] & a[n,m] \end{bmatrix}$$

which has n rows and m columns.

Example

The following is a 4 * 5 matrix:

$$\begin{bmatrix} 12 & 8 & 9 & 7 & 6 \\ 7 & 6 & 89 & 56 & 2 \\ 5 & 5 & 6 & 9 & 10 \\ 8 & 6 & 0 & -8 & -1 \end{bmatrix}$$

Review of Matrix Multiplication

The product $C = AB$ of a $p * q$ matrix A and a $q * r$ matrix B is a $p * r$ matrix given by

$$c[i,j] = \sum_{k=1}^{q} a[i,k]b[k,j], \text{ for } 1 \leq i \leq p \text{ and } 1 \leq j \leq r$$

Complexity of Matrix multiplication:Note that C has pr entries and each entry takes $\Theta(q)$ time to compute so the total procedure takes $\Theta(pqr)$ time.

Example

$$A = \begin{bmatrix} 1 & 8 & 9 \\ 7 & 6 & -1 \\ 5 & 5 & 6 \end{bmatrix}, B = \begin{bmatrix} 1 & 8 \\ 7 & 6 \\ 5 & 5 \end{bmatrix}, C = AB = \begin{bmatrix} 102 & 101 \\ 44 & 87 \\ 70 & 100 \end{bmatrix}$$

Remarks on Matrix Multiplication

- Multiplication is recursively defined by

$$A_1 A_2 A_3 ... A_{s-1} A_s = A_1(A_2(A_3 ...(A_{s-1} A_s)))$$

- Matrix multiplication is associative, e.g.,

$$A_1 A_2 A_3 = (A_1 A_2) A_3 = A_1(A_2 A_3)$$

so parenthesization does not change result.

Matrix Multiplication of ABC

- Given a $p * q$ matrix A and a $q * r$ matrix B and a $r * s$ matrix C, then ABC can be computed in two ways $(AB)C$ and $A(BC)$
- The number of multiplications needed are:

$$mult[(AB)C] = pqr + prs$$

$$mult[A(BC)] = qrs + pqs$$

Example

When $p = 5$, $q = 4$, $r = 6$, and $s = 2$, then

$$mult[(AB)C] = 120 + 60 = 180$$

$$mult[A(BC)] = qrs + pqs = 48 + 44 = 88$$

A big difference!

Implication: The multiplication "sequence" (parenthesization) is important!!

Outline

The Chain Matrix Multiplication Problem

Definition (Chain matrix multiplication problem)

Given dimensions $p_0, p_1, ..., p_n$, corresponding to matrix sequence $A_1, A_2, ..., A_n$ where A_i has dimension $p_{i-1} * p_i$ determine the "multiplication sequence" that minimizes the number of scalar multiplications in computing $A_1 A_2 ... A_n$

- i.e., determine how to parenthesize the multiplications.

Example

$$A_1 A_2 A_3 A_4 = (A_1 A_2)(A_3 A_4) = A_1(A_2(A_3 A_4)) = A_1((A_2 A_3)A_4)$$
$$= ((A_1 A_2)A_3)A_4 = (A_1(A_2 A_3))A_4$$

Exhaustive search: $\Omega(4^n/n^{3/2})$

Question

Any better approach?

Yes - Dynamic Programming

Outline

Developing a Dynamic Programming Algorithm(Step 1)

Step 1: Space of subproblems

- For each pair $1 \leq i \leq j \leq n$, determine the multiplication sequence for $A_{i...j} = A_i A_{i+1} ... A_j$ that minimizes the number of multiplications.
- Clearly, $A_{i...j}$ is a $p_{i-1} * p_j$ matrix.
- Original Problem: determine sequence of multiplication for $A_{1...n}$.

Relationships among subproblems

For any optimal multiplication sequence, at the last step you are multiplying two matrices $A_{i\ldots k}$ and $A_{k+1\ldots j}$ for some k. That is,

$$A_{i\ldots j} = (A_i \ldots A_k)(A_{k+1} \ldots A_j) = A_{i\ldots k} A_{k+1\ldots j}$$

Question

How do we decide where to split the chain (what is k)?

ANS: Need to search all possible values of k

Question

How do we parenthesize the subchains $A_{i\ldots k}$ and $A_{k+1\ldots j}$?

ANS: $A_{i\ldots k}$ and $A_{k+1\ldots j}$ must be optimal so we can apply the same procedure recursively

Optimal structure Property

If the final "optimal" solution of $A_{i...j}$ involves splitting into $A_{i...k}$ and $A_{k+1...j}$ at the final step, then parenthesization of $A_{i...k}$ and $A_{k+1...j}$ j in the final optimal solution must also be optimal for the subproblems.

- If parenthesization of $A_{i...k}$ was not optimal, we could replace it by a better parenthesization and get a cheaper final solution, leading to a contradiction.
- Similarly, if parenthesization of $A_{k+1...j}$ was not optimal, we could replace it by a better parenthesization and get a cheaper final solution, leading to a contradiction.

Developing a Dynamic Programming Algorithm(Step 2)

Step 2: Value of an problem and those of its subproblems

- For $1 \leq i \leq j \leq n$, let $m[i,j]$ denote the minimum number of multiplications needed to compute $A_{i\ldots j}$. The optimum cost can be described by the following recursive definition.

$$m[i,j] = \begin{cases} 0 & i = j, \\ min_{i \leq k < j}(m[i,k] + m[k+1,j] + p_{i-1}p_k p_j) & i < j \end{cases}$$

Proof.

Any optimal sequence of multiplication for $A_{i\ldots j}$ is equivalent to some choice of splitting $A_{i\ldots j} = A_{i\ldots k}A_{k+1\ldots j}$ for some k, where the sequences of multiplications for $A_{i\ldots k}$ and $A_{k+1\ldots j}$ are also optimal. Hence

$$m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_k p_j$$

We don't know what k is, though.
But, there are only $j - i$ possible values of k so we can check them all and find the one which returns a smallest cost. Therefore

$$m[i,j] = \begin{cases} 0 & i = j, \\ min_{i \le k < j}(m[i,k] + m[k+1,j] + p_{i-1}p_k p_j) & i < j \end{cases}$$

Developing a Dynamic Programming Algorithm(Step 3)

Step 3: Bottom-up computation of $m[i, j]$

Recurrence:

$$m[i, j] = min_{i \leq k < j}(m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j)$$

Compute and save $m[i, j]$ in such an order that when it is time to calculate $m[i, j]$, the values of $m[i, k]$ and $m[k + 1, j]$ are already available.

Compute them in increasing order of the length of the matrix-chain:

$m[1, 2], m[2, 3], m[3, 4], ...m[n - 3, n - 2], m[n - 2, n - 1], m[n - 1, n]$

$m[1, 3], m[2, 4], m[3, 5], ...m[n - 3, n - 1], m[n - 2, n]$

$m[1, 4], m[2, 5], m[3, 6], ...m[n - 3, n]$

...

$m[1, n - 1], m[2, n]$

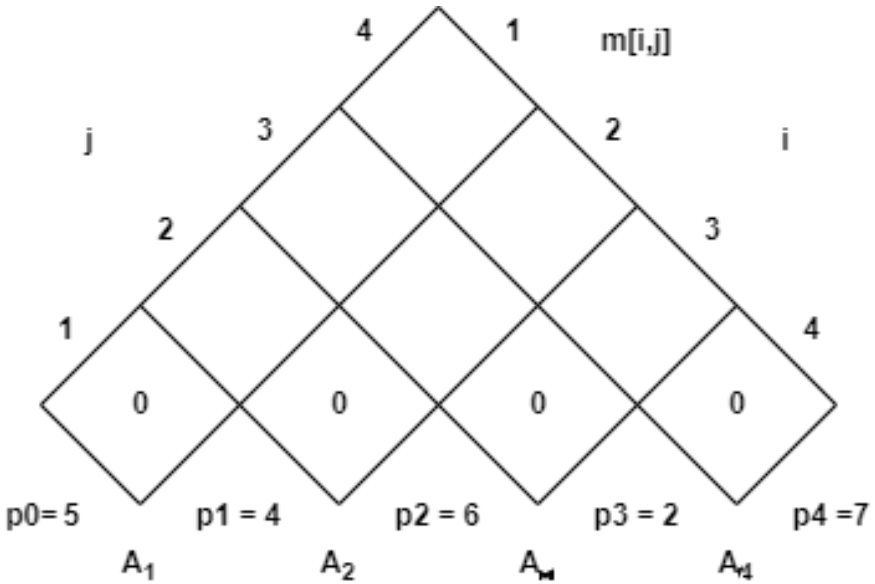
$m[1, n]$

Example for the Bottom-Up Computation I

Example

Given a chain of four matrices A_1, A_2, A_3, and A_4 with $p_0 = 5$, $p_1 = 4$, $p_2 = 6$, $p_3 = 2$, and $p_4 = 7$. Find $m[1, 4]$.

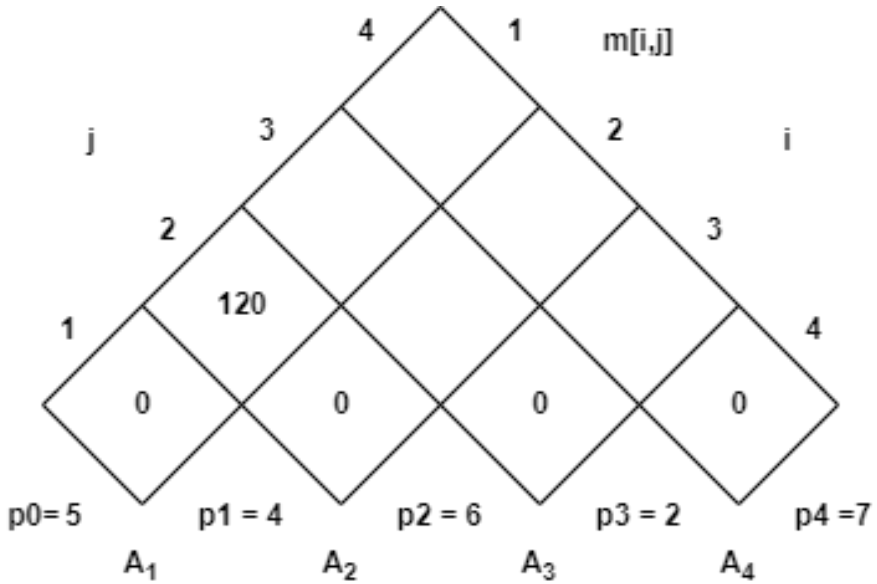
0. Initialization



Example for the Bottom-Up Computation II

1. Computing $m[1,2]$. By definition

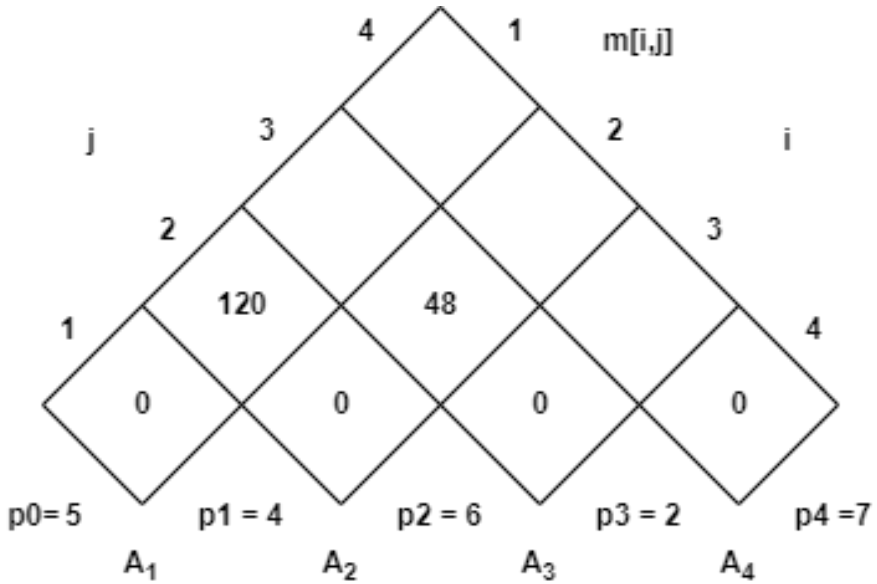
$$\begin{aligned} m[1,2] &= min_{1 \leq k < 2}(m[1,k] + m[k+1,2] + p_0 p_k p_2) \\ &= m[1,1] + m[2,2] + p_0 p_1 p_2 = 120 \end{aligned}$$



Example for the Bottom-Up Computation III

2. Computing $m[2,3]$. By definition

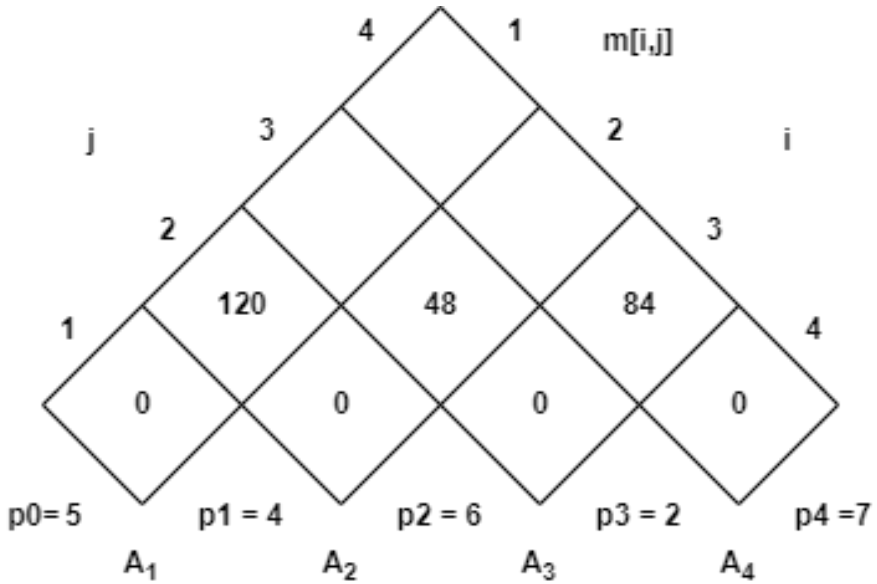
$$\begin{aligned} m[2,3] &= min_{2 \leq k < 3}(m[2,k] + m[k+1,3] + p_1 p_k p_3) \\ &= m[2,2] + m[3,3] + p_1 p_2 p_3 = 48 \end{aligned}$$



Example for the Bottom-Up Computation IV

3. Computing $m[3,4]$. By definition

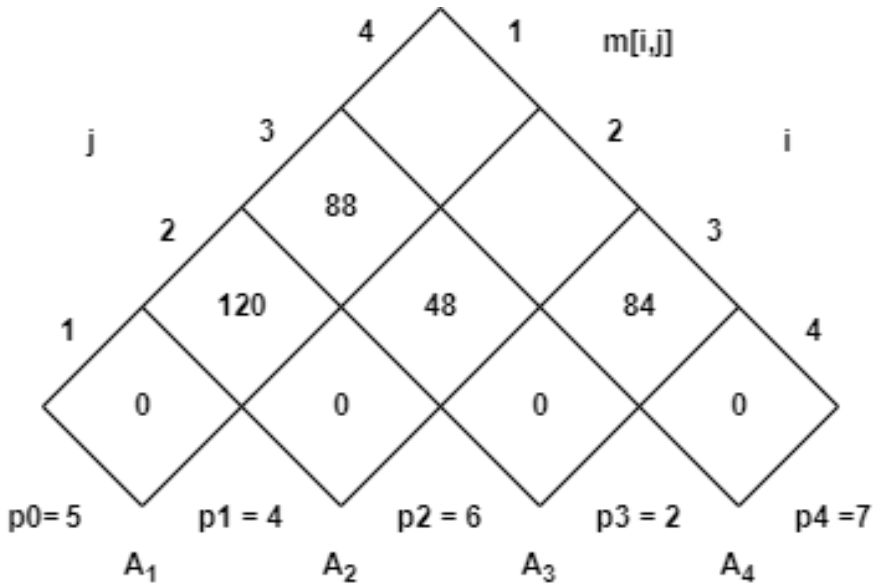
$$\begin{aligned} m[3,4] &= min_{3 \leq k < 4}(m[3,k] + m[k+1,4] + p_2 p_k p_4) \\ &= m[3,3] + m[4,4] + p_2 p_3 p_4 = 84 \end{aligned}$$



Example for the Bottom-Up Computation V

4. Computing $m[1,3]$. By definition

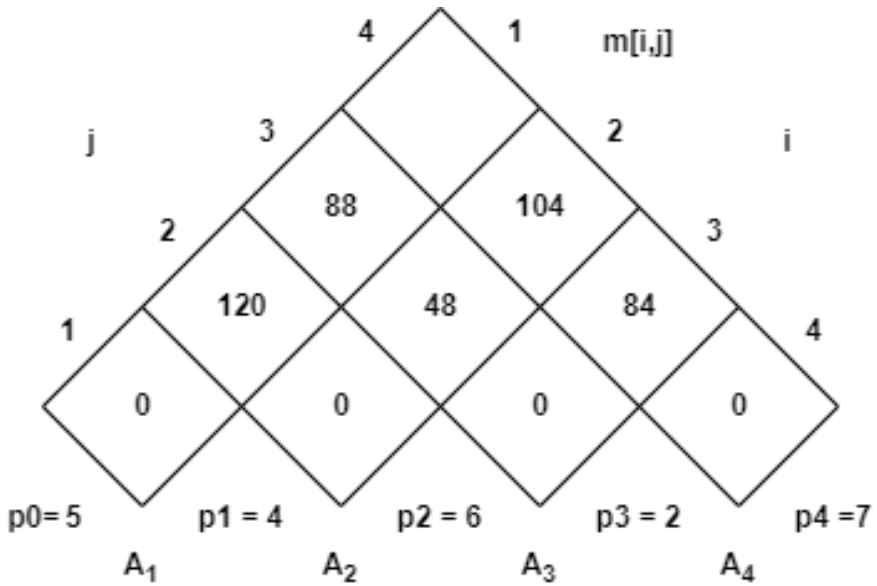
$$\begin{aligned} m[1,3] &= min_{1 \leq k < 3}(m[1,k] + m[k+1,3] + p_0 p_k p_3) \\ &= min \left\{ \begin{array}{l} m[1,1] + m[2,3] + p_0 p_1 p_3 \\ m[1,2] + m[3,3] + p_0 p_2 p_3 \end{array} \right\} \\ &= 88 \end{aligned}$$



Example for the Bottom-Up Computation VI

5. Computing $m[2,4]$. By definition

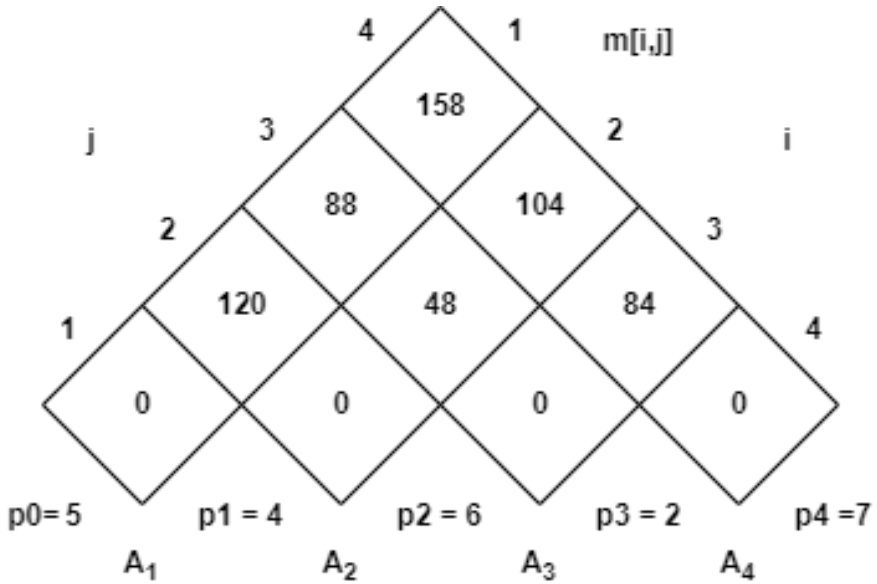
$$\begin{aligned} m[2,4] &= min_{2\leq k<4}(m[2,k]+m[k+1,4]+p_1p_kp_4) \\ &= min\left\{\begin{array}{l} m[2,2]+m[3,4]+p_1p_2p_4 \\ m[2,3]+m[4,4]+p_1p_3p_4 \end{array}\right\} \\ &= 104 \end{aligned}$$



Example for the Bottom-Up Computation VII

6. Computing $m[1,4]$. By definition

$$
\begin{aligned}
m[1,4] &= min_{1 \leq k < 4}(m[1,k] + m[k+1,4] + p_0 p_k p_4) \\
&= min \begin{cases} m[1,1] + m[2,4] + p_0 p_1 p_4 \\ m[1,2] + m[3,4] + p_0 p_2 p_4 \\ m[1,3] + m[4,4] + p_0 p_3 p_4 \end{cases} \\
&= 158
\end{aligned}
$$



Developing a Dynamic Programming Algorithm(Step 4)

Step 4: Constructing optimal solution

Idea: Maintain an array $s[1...n, 1...n]$, where $s[i, j]$ denotes k for the optimal splitting in computing $A_{i...j} = A_{i...k}A_{k+1...j}$.

Question

How to Recover the Multiplication Sequence using $s[1...n, 1...n]$?

$s[1, n]$ $\quad (A_1...A_{s[1,n]})(A_{s[1,n]+1}...An)$

$s[1, s[1, n]]$ $\quad (A_1...A_{s[1,s[1,n]]})(A_{s[1,s[1,n]]+1}...A_{s[1,n]})$

$s[s[1, n] + 1, n]$ $\quad (A_{s[1,n]+1}...A_{s[s[1,n]+1,n]})(A_{s[s[1,n]+1,n]+1}...A_n)$

... $\quad$ ...

Do this recursively until the multiplication sequence is determined.

Developing a Dynamic Programming Algorithm(Step 4)

Example

Finding the Multiplication Sequence

Consider $n = 6$. Assume that the array $S[1...6, 1...6]$ has been computed. The multiplication sequence is recovered as follows.

$$\begin{aligned} s[1,6] &= 3 \quad (A_1A_2A_3)(A_4A_5A_6) \\ s[1,3] &= 1 \quad (A_1(A_2A_3)) \\ s[4,6] &= 5 \quad ((A_4A_5)A_6) \end{aligned}$$

Hence the final multiplication sequence is

$$(A_1(A_2A_3))((A_4A_5)A_6)$$

The Dynamic Programming Algorithm

Algorithm 1 Matrix-Chain(p, n)

```
1:  for i = 1 to n do
2:      m[i, i] = 0
3:  end for
4:  for l = 2 to n do      // l is length of sub-chain
5:      for i = 1 to n − l + 1 do
6:          j = i + l − 1
7:          m[i, j] = ∞
8:          for k = i to j − 1 do
9:              q = m[i, k] + m[k + 1, j] + p[i − 1] ∗ p[k] ∗ p[j]
10:             if q < m[i, j] then
11:                 m[i, j] = q
12:                 s[i, j] = k
13:             end if
14:         end for
15:     end for
16: end for
17: return m and s      // (Optimum in m[1, n])
```

Complexity:The loops are nested three levels deep. Each loop index takes on $\leq n$ values. Hence the time complexity is $\mathcal{O}(n^3)$. Space complexity is $\Theta(n^2)$.